Logistic Regression Weight Optimization

Optimization Theory

2025. 11. 강윤의

Step 1. 준비 및 수학적 모델 정의

1. 토이 프로젝트 데이터 생성

2D 평면에 두 클래스(0과 1)의 데이터를 생성

- Class 0: (1, 1) 근처에 15개 점 ([1, 1] + N(0, 0.5)).
- Class 1: (3, 3) 근처에 15개 점 ([3, 3] + N(0, 0.5)).

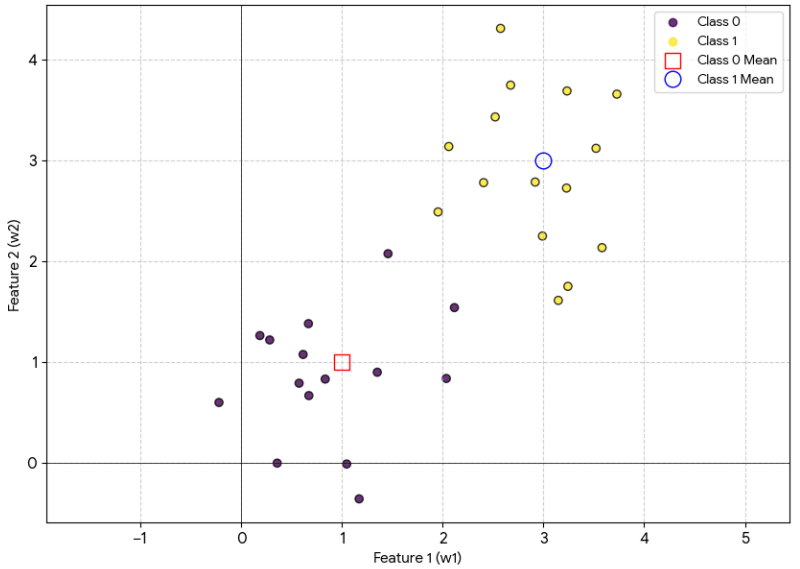
데이터 $\boldsymbol{X}$ (30x2 행렬)와 라벨 $\boldsymbol{y}$ (30x1 벡터)를 준비

```python
def generate_toy_data(n_samples_per_class=15, mean_0=[1, 1], mean_1=[3, 3], cov=0.5):
    cov_matrix = np.array([[cov, 0], [0, cov]])
    X_0 = np.random.multivariate_normal(mean_0, cov_matrix, n_samples_per_class)
    y_0 = np.zeros(n_samples_per_class)
    X_1 = np.random.multivariate_normal(mean_1, cov_matrix, n_samples_per_class)
    y_1 = np.ones(n_samples_per_class)

    X = np.vstack((X_0, X_1))
    y = np.hstack((y_0, y_1))

    X_b = np.hstack((X, np.ones((X.shape[0], 1))))

    return X, X_b, y
```



Step 1. 준비 및 수학적 모델 정의

2. 목적 함수 및 제약 조건

변수: $w = [w_1, w_2, w_b]^T$ (3차원 벡터)

목적 함수 $f(w)$**:** Negative Log-Likelihood

$$f(w) = -\sum_{i=1}^{30} \left[y_i \log\left(\sigma\left(\mathrm{x}_i^T w\right)\right) + (1 - y_i) \log\left(1 - \sigma\left(\mathrm{x}_i^T w\right)\right)\right]$$

(여기서 $\mathrm{x}_i = \left[x_{\{i,1\}}, x_{\{i,2\}}, 1\right]^T$ 이고, σ는 시그모이드 함수)

제약 조건

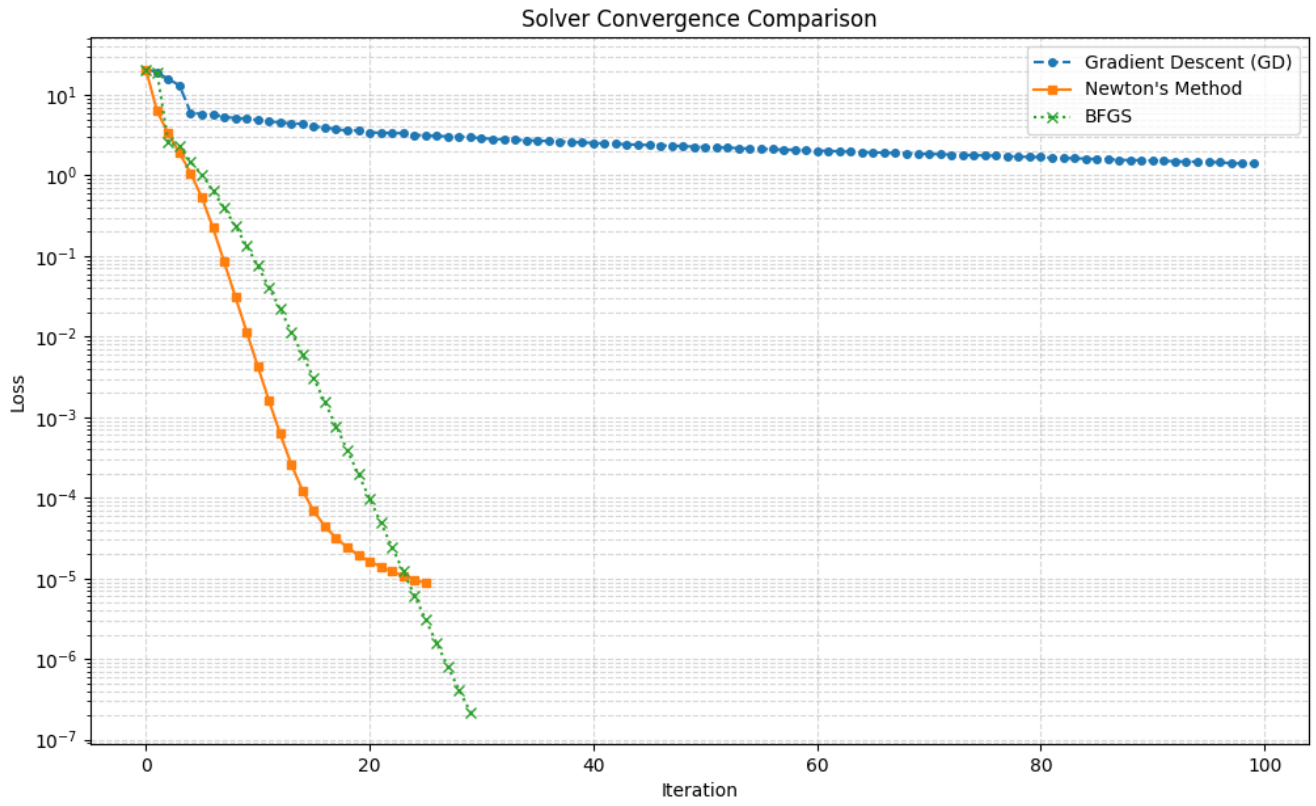
- $g_1(w) = -w_1 \leq 0$ 가중치가 양수여야 함
- $g_2(w) = -w_2 \leq 0$

```python
def calculate_loss(w, X_b, y):
    N = len(y)
    z = X_b @ w
    p = sigmoid(z)

    epsilon = 1e-9
    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -np.sum(y * np.log(p) + (1 - y) * np.log(1 - p))
    return loss
```

Step 2. 알고리즘 개발 및 비교




```
GD     | Loss: 1.407337 | w: [  2.59705086   2.52300723 -10.01470985]
Newton | Loss: 0.000009 | w: [  24.66210599   31.48846334 -114.04497538]
BFGS   | Loss: 0.000000 | w: [  48.69109757   54.18943843 -214.68724195]
```

- **GD:** 그래디언트만 사용하기 때문에, 100번의 Iteration 동안 Loss이 아주 느리게 (선형적으로) 감소
- **Newton:** 헤시안을 사용해 그래프가 거의 수직으로 떨어짐. 이는 최적해 근처에서 Quadratic Convergence 속도를 갖는 뉴턴법의 강력한 특징
- **BFGS:** 최종 손실이 0으로, 가장 낮은 손실에 도달

Step 3. 제약 조건 솔버 설계

1. Barrier Method Architecture

$$p_{\text{barrier}}(\mathbf{x}) = -\sum_i \begin{cases} \log(-g_i(\mathbf{x})) & \text{if } g_i(\mathbf{x}) \geq -1 \\ 0 & \text{otherwise} \end{cases}$$

$$L(w, \rho) = f(w) - (1/\rho) \times (\log(w_1) + \log(w_2))$$

w_1이 0에 가까워지면(예: 0.1, 0.01…), $\log(w_1)$ 값은 $-\infty$
$-(1/\rho)$가 붙으면 이 항 전체는 $+\infty$
▶ 솔버는 $L(w, \rho)$의 값을 최소화하려 하기 때문에, 이 값이 무한대가 되는 $w_1 = 0$ 근처로는 가지 않으려 함.

$$\boldsymbol{f(w) \leftarrow L(w, \rho)}$$

```python
def calculate_barrier_loss(w, X_b, y, rho):
    if w[0] <= 0 or w[1] <= 0:
        return np.inf

    f_loss = calculate_loss(w, X_b, y)
    barrier_term = -(1.0 / rho) * (np.log(w[0]) + np.log(w[1])) #
    return f_loss + barrier_term

def calculate_barrier_gradient(w, X_b, y, rho):
    f_grad = calculate_gradient(w, X_b, y)
    barrier_grad = -(1.0 / rho) * np.array([1.0 / w[0], 1.0 / w[1], 0.0])
    return f_grad + barrier_grad

def calculate_barrier_hessian(w, X_b, y, rho):
    f_hessian = calculate_hessian(w, X_b, y)
    barrier_hessian_diag = (1.0 / rho) * np.array([1.0 / (w[0]**2), 1.0 / (w[1]**2), 0.0])
    return f_hessian + np.diag(barrier_hessian_diag)
```

```python
def constrained_line_search(w, d, X_b, y, rho, alpha_start=1.0, rho_ls=0.5, beta=1e-4):
    alpha = alpha_start

    while True:
        w_next = w + alpha * d
        if w_next[0] > 0 and w_next[1] > 0:
            break
        alpha = alpha * rho_ls
        if alpha < 1e-12:
            return 0.0

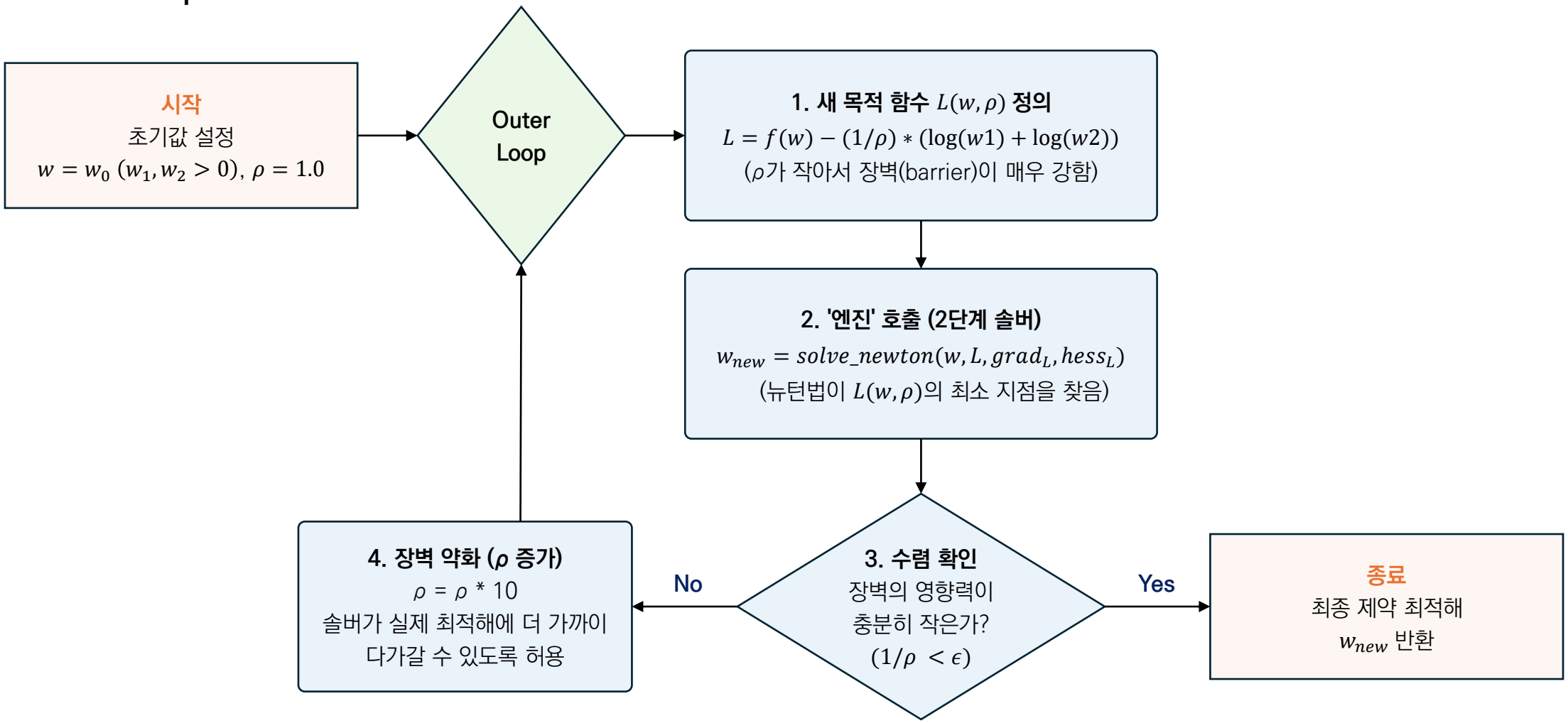
    current_loss = calculate_barrier_loss(w, X_b, y, rho)
    grad = calculate_barrier_gradient(w, X_b, y, rho)
    grad_dot_d = grad @ d

    while calculate_barrier_loss(w + alpha * d, X_b, y, rho) > current_loss + beta * alpha * grad_dot_d:
        alpha = alpha * rho_ls
        if alpha < 1e-12:
            return 0.0

    return alpha
```

Step 3. 제약 조건 솔버 설계

2. Outer Loop



[실제 프로젝트 적용]

UCI Breast Cancer dataset binary classification

Toy project	Real project
Data (30, 3) 행렬	Data (569, 31) 행렬
제약 조건 $-w_1, w_2 \leq 0$ $L = f(w) - (1/\rho) * (\log(w1) + \log(w2))$	$L = f(w) - (1/\rho)(\sum_{i \in Group1} \log(w_i) + \sum_{j \in Group2} \log(-w_j))$
if w_next[0] 〉 ϵ and w_next[1] 〉 ϵ: break	w_next[group1_indices] 〉 ϵ w_next[group2_indices] 〈 $-\epsilon$
mu[0] = g_star[0], mu[1] = g_star[1]	Group 1 $(g_i = -w_i \leq 0): \nabla g_i = [\dots, -1, \dots]$ Stationarity: $\nabla f_i + \mu_i(-1) = 0 \rightarrow \mu_i = \nabla f_i$ Dual Feasibility: $\mu_i = \nabla f_i \geq 0$ 인지 확인 Group 2 $(g_i = w_j \leq 0): \nabla g_j = [\dots, +1, \dots]$ Stationarity: $\nabla f_j + \mu_j(1) = 0 \rightarrow \mu_j = -\nabla f_i$ Dual Feasibility: $\mu_j = -\nabla f_j \geq 0$ 인지 확인. Group 3 (제약 없음): $\mu_k = 0$ Stationarity: $\nabla f_k = 0$ 인지 확인.

Step 1. 준비 및 모델 재정의

UCI Breast Cancer dataset 전처리 및 제약 조건 설계

데이터: N개의 데이터 샘플 (Train set = 455개), 각 샘플은 30개의 feature을 가짐

변수: $w = [w_1, w_2, \ldots, w_{30}, w_b]^T$ (31차원 벡터)

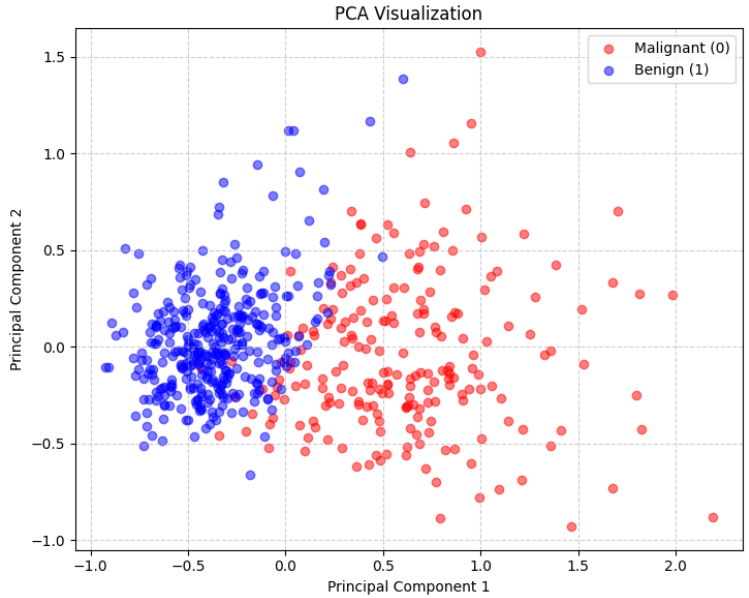
목적 함수 $f(w)$: Negative Log-Likelihood

$$f(w) = -\sum_{i=1}^{N} \left[y_i \log\left(\sigma\left(\mathrm{x}_i^T w\right)\right) + (1 - y_i) \log\left(1 - \sigma\left(\mathrm{x}_i^T w\right)\right)\right]$$

(여기서 $\mathrm{x}_i = \left[x_{\{i,1\}}, x_{\{i,2\}}, 1\right]^T$ 이고, σ는 시그모이드 함수)

제약 조건

- Group 1: 클수록 나쁜 (악성) 특성들: $g_i(w) = w_i \leq 0$
- Group 2: 클수록 좋은 (양성) 특성들: $g_j(w) = -w_j \leq 0$
- Group 3: 관계 불명확 특성들: Unconstrained



Group 1: 종양 크기(x)가 커지면 → 양성(1)일 확률(p)은 낮아져야 함.
▶$p = \sigma(w \cdot x)$ 에서 p가 작아지려면 $w \cdot x$가 작아져야 함.
▶$x \geq 0$으로 스케일링 되었으므로, w는 반드시 음수.

Group 2: 표면이 매끄러우면(x) → 양성(1)일 확률(p)은 높아짐.
▶$p = \sigma(w \cdot x)$ 에서 p가 커지려면 $w \cdot x$가 커져야 함.
▶$x \geq 0$으로 스케일링 되었으므로, w는 반드시 양수.

Step 1. 준비 및 모델 재정의

UCI Breast Cancer dataset 전처리 및 제약 조건 설계

```python
def load_and_preprocess_data():
    data = load_breast_cancer()
    X_raw = data.data
    y_raw = data.target
    feature_names = data.feature_names

    print("--- 1. Data Loading ---")
    print(f"Original X shape: {X_raw.shape}")
    print(f"Original y shape: {y_raw.shape}")

    scaler = MinMaxScaler()
    X_scaled = scaler.fit_transform(X_raw)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y_raw, test_size=0.2, rando
    )

    X_b_train = np.hstack((X_train, np.ones((X_train.shape[0], 1))))
    X_b_test = np.hstack((X_test, np.ones((X_test.shape[0], 1))))

    return X_b_train, y_train, X_b_test, y_test, feature_names
```

데이터 스케일링

테스트셋 분할

```python
def define_constraint_groups(feature_names):
    feature_list = list(feature_names)
    group1_names = [
        'mean radius', 'mean perimeter', 'm
        'mean concavity', 'mean concave points',
        'radius error', 'perimeter error', 'area error',
        'concavity error', 'concave points error',
        'worst radius', 'worst perimeter', 'worst area',
        'worst concavity', 'worst concave points'
    ]
    group2_names = [
        'mean smoothness', 'mean compactness', 'mean symmetry',
        'smoothness error', 'compactness error', 'symmetry error',
        'worst smoothness', 'worst compactness', 'worst symmetry'
    ]
    group3_names = [
        'mean texture', 'mean fractal dimension',
        'texture error', 'fractal dimension error',
        'worst texture', 'worst fractal dimension'
    ]

    group1_indices = [feature_list.index(name) for name in group1_names]
    group2_indices = [feature_list.index(name) for name in group2_names]
    group3_indices = [feature_list.index(name) for name in group3_names]

    bias_index = 30
    group3_indices.append(bias_index)

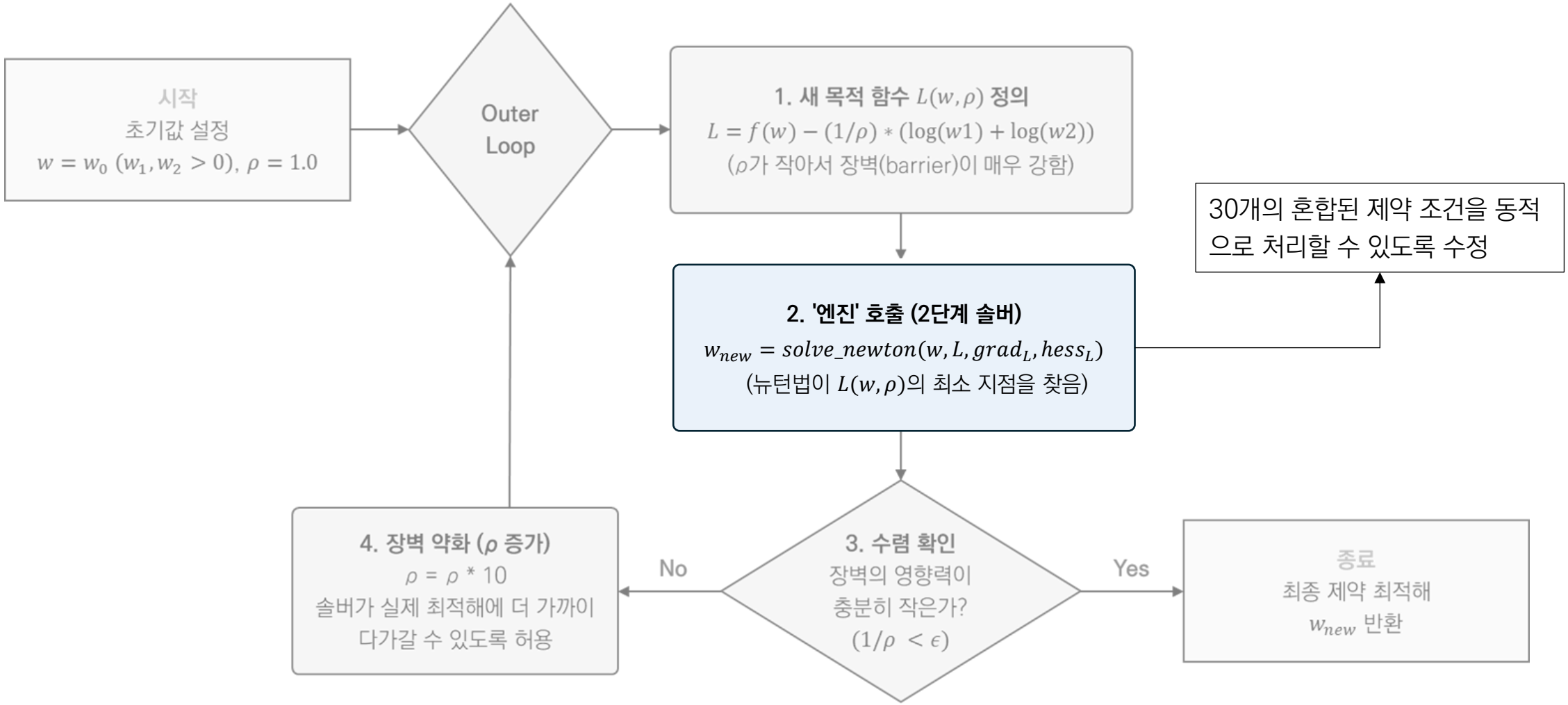
    return group1_indices, group2_indices, group3_indices

def initialize_feasible_w0(n_features, group1_indices, group2_indices):
    w0 = np.zeros(n_features)
    w0[group1_indices] = -0.1
    w0[group2_indices] = 0.1
    return w0
```

제약 조건 설계

초기값 생성

Step 2. 솔버 엔진 일반화



Step 2. 솔버 엔진 일반화

Barrier Method Architecture

$$L = f(w) - (1/\rho)\left(\sum_{i \in Group1} \log(w_i) + \sum_{j \in Group2} \log(-w_j)\right)$$

- Group 1 ($w_i \le 0$): $-\frac{1}{\rho}\log(-w_i)$
- Group 2 ($w_j \ge 0$): $-\frac{1}{\rho}\log(w_j)$

```python
def calculate_barrier_loss(w, X_b, y, rho, group1_indices, group2_indices):
    epsilon = 1e-12

    if np.any(w[group1_indices] > -epsilon):
        return np.inf
    if np.any(w[group2_indices] < epsilon):
        return np.inf

    f_loss = calculate_loss(w, X_b, y)
    barrier_sum = 0.0

    barrier_sum += np.sum(np.log(-w[group1_indices]))
    barrier_sum += np.sum(np.log(w[group2_indices]))

    total_loss = f_loss - (1.0 / rho) * barrier_sum
    return total_loss
```

```python
def calculate_barrier_gradient(w, X_b, y, rho, group1_indices, group2_indices):
    grad = calculate_gradient(w, X_b, y)

    grad[group1_indices] -= 1.0 / (rho * w[group1_indices])
    grad[group2_indices] -= 1.0 / (rho * w[group2_indices])

    return grad

def calculate_barrier_hessian(w, X_b, y, rho, group1_indices, group2_indices):
    H = calculate_hessian(w, X_b, y)

    barrier_diag = np.zeros_like(w)
    barrier_diag[group1_indices] = 1.0 / (rho * w[group1_indices]**2)
    barrier_diag[group2_indices] = 1.0 / (rho * w[group2_indices]**2)
    H += np.diag(barrier_diag)

    return H
```

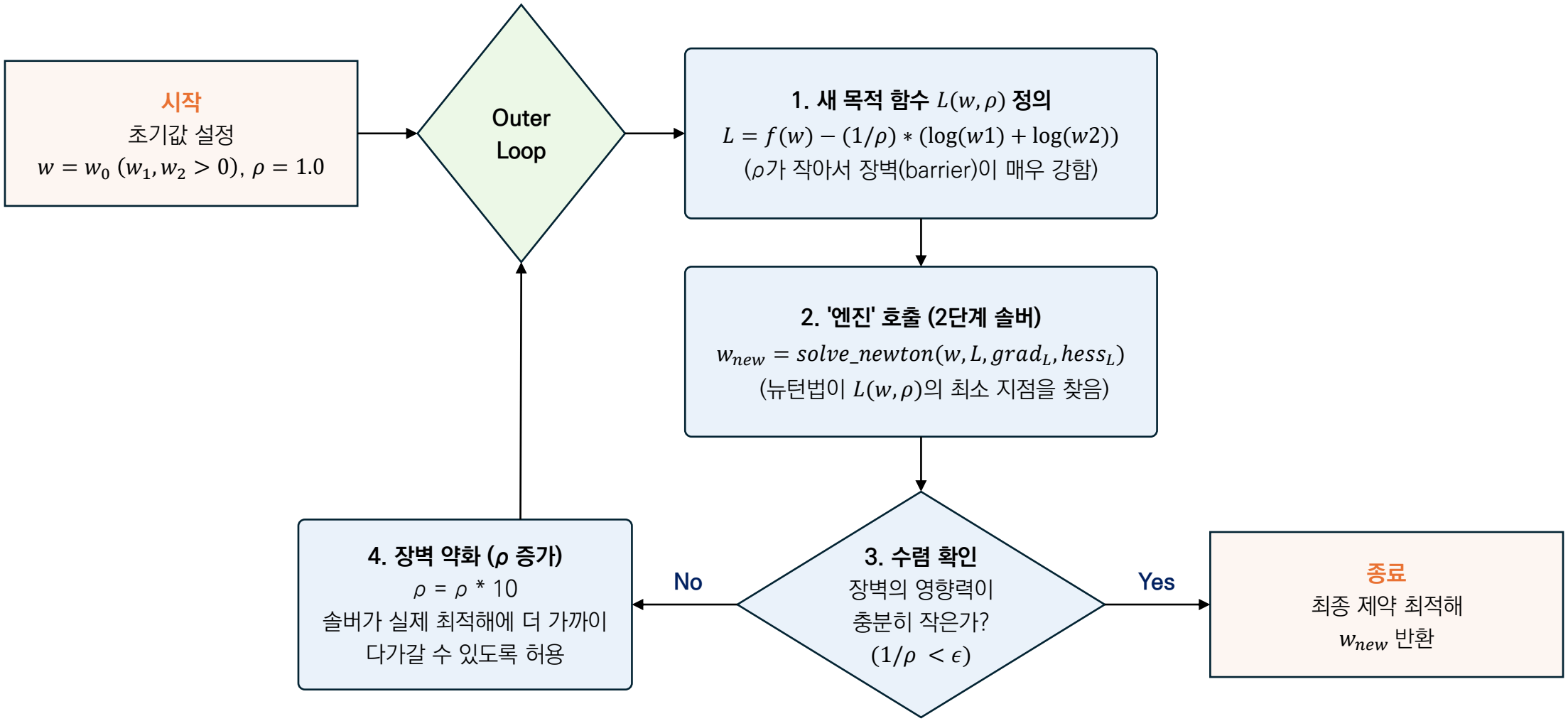
Line Search (Feasibility Check)

w_next[group1_indices] $\langle -\epsilon$
w_next[group2_indices] $\rangle \epsilon$

```python
def constrained_line_search(w, d, X_b, y, rho, group1_indices, group2_indices, alpha_start=1.0, rho_ls=0.5, beta=1e-4):
    alpha = alpha_start
    epsilon = 1e-12
    max_ls_iter = 50

    for _ in range(max_ls_iter):
        w_next = w + alpha * d
        feasible_g1 = np.all(w_next[group1_indices] < -epsilon)
        feasible_g2 = np.all(w_next[group2_indices] > epsilon)
```

Step 3. 제약 솔버 실행



Step 3. 제약 솔버 실행

Outer Iter	Rho	Tolerance	Loss	Avg G1(Neg)	Avg G2(Pos)
0	1	2.40e+01	36.3813	-23.4353	7.9642
1	10	2.40e+00	23.3588	-12.2808	3.1578
2	100	2.40e-01	22.2741	-12.0746	2.7627
3	1000	2.40e-02	22.1497	-12.1736	2.7146
4	10000	2.40e-03	22.1365	-12.2095	2.7071
5	100000	2.40e-04	22.1352	-12.2158	2.7062
6	1000000	2.40e-05	22.1351	-12.2165	2.7061
7	10000000	2.40e-06	22.135	-12.2166	2.7061
8	100000000	2.40e-07	22.135	-12.2166	2.7061

0	mean radius	G1(<=0)	-0.000000	-0.05019374
1	mean texture	G3(None)	5.573114	0.00000000
2	mean perimeter	G1(<=0)	-0.000000	-0.04657385
3	mean area	G1(<=0)	-0.000000	-0.04395221
4	mean smoothness	G2(>=0)	0.000000	0.18858824
5	mean compactness	G2(>=0)	22.411287	0.00000000
6	mean concavity	G1(<=0)	-4.983287	-0.00000000
7	mean concave points	G1(<=0)	-18.118714	-0.00000000
8	mean symmetry	G2(>=0)	0.000000	0.46730130
9	mean fractal dimension	G3(None)	-2.757133	0.00000000
10	radius error	G1(<=0)	-0.000018	-0.00054372
11	texture error	G3(None)	13.347148	0.00000000
12	perimeter error	G1(<=0)	-0.000000	-0.02019288
13	area error	G1(<=0)	-109.903542	-0.00000000
14	smoothness error	G2(>=0)	0.000000	0.22479873
15	compactness error	G2(>=0)	1.943262	0.00000001
16	concavity error	G1(<=0)	-0.000000	-0.11036878
17	concave points error	G1(<=0)	-16.357372	-0.00000000
18	symmetry error	G2(>=0)	0.000000	0.22451841
19	fractal dimension error	G3(None)	33.345284	0.00000000
20	worst radius	G1(<=0)	-22.284088	-0.00000000
21	worst texture	G3(None)	-24.589247	0.00000000
22	worst perimeter	G1(<=0)	-0.000006	-0.00157660
23	worst area	G1(<=0)	-0.000004	-0.00223872
24	worst smoothness	G2(>=0)	0.000000	0.36000519
25	worst compactness	G2(>=0)	0.000000	0.07815570
26	worst concavity	G1(<=0)	-3.214993	-0.00000000
27	worst concave points	G1(<=0)	-8.386738	-0.00000000
28	worst symmetry	G2(>=0)	0.000000	0.58421626
29	worst fractal dimension	G3(None)	-24.853249	0.00000000

w^* = [-12.2, 2.7, 26.38] (average)

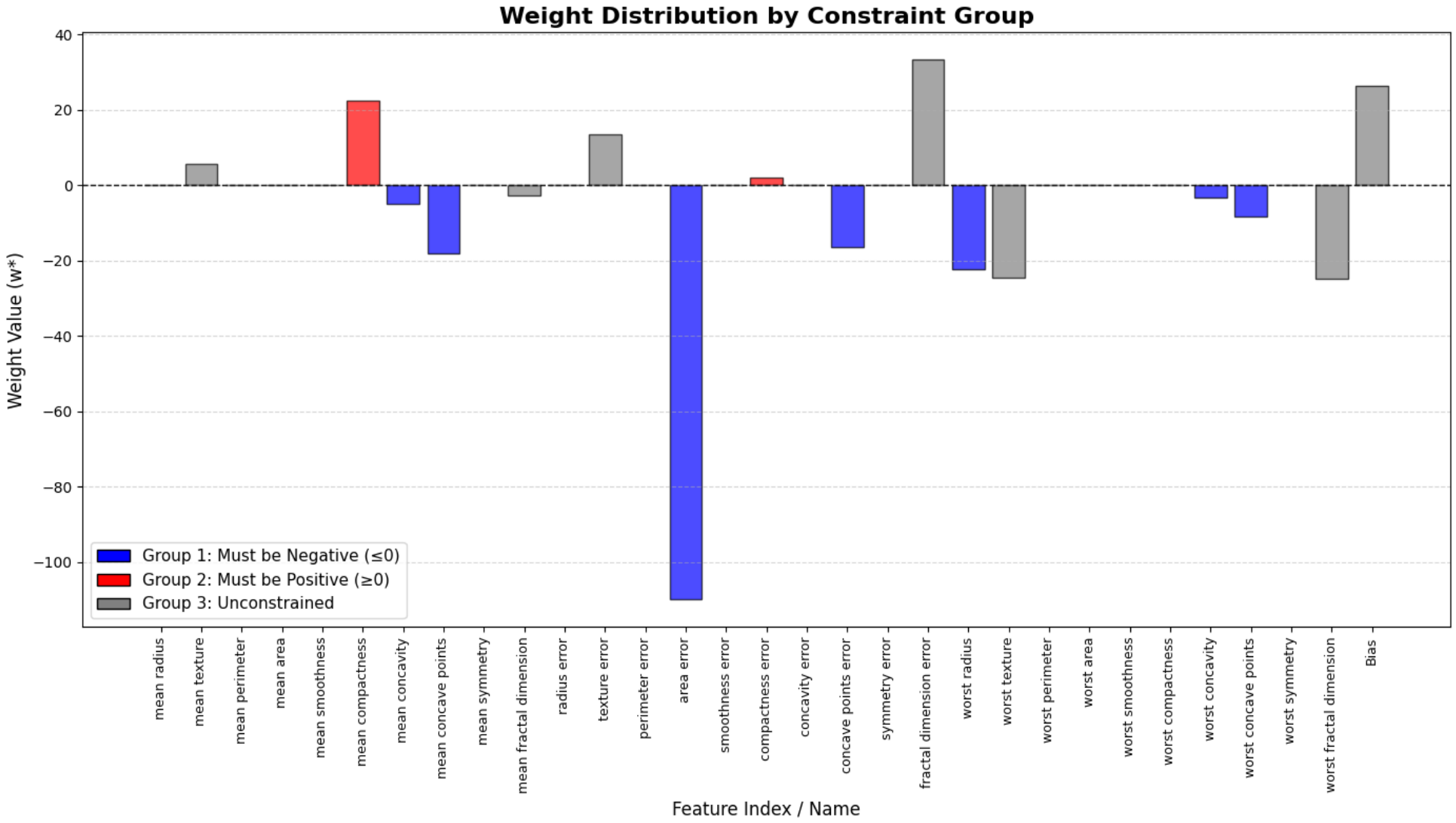
최종 Loss: 22.135

제약 조건 만족 – **Group 1** $(w_i \leq 0)$: Max value = -9.06e-08 : True

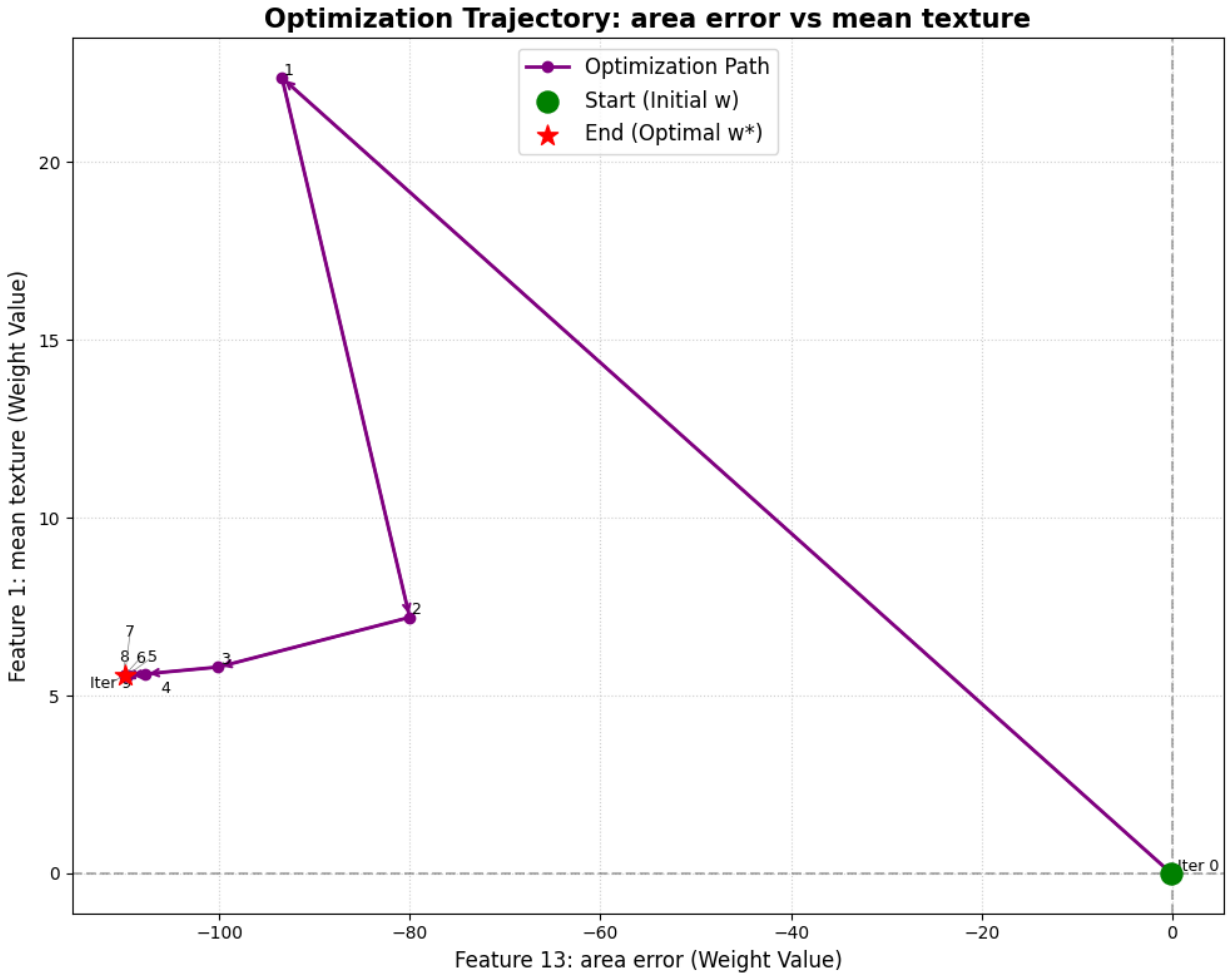
– **Group 2** $(w_j \geq 0)$: Min Value = 1.71e-08 : True

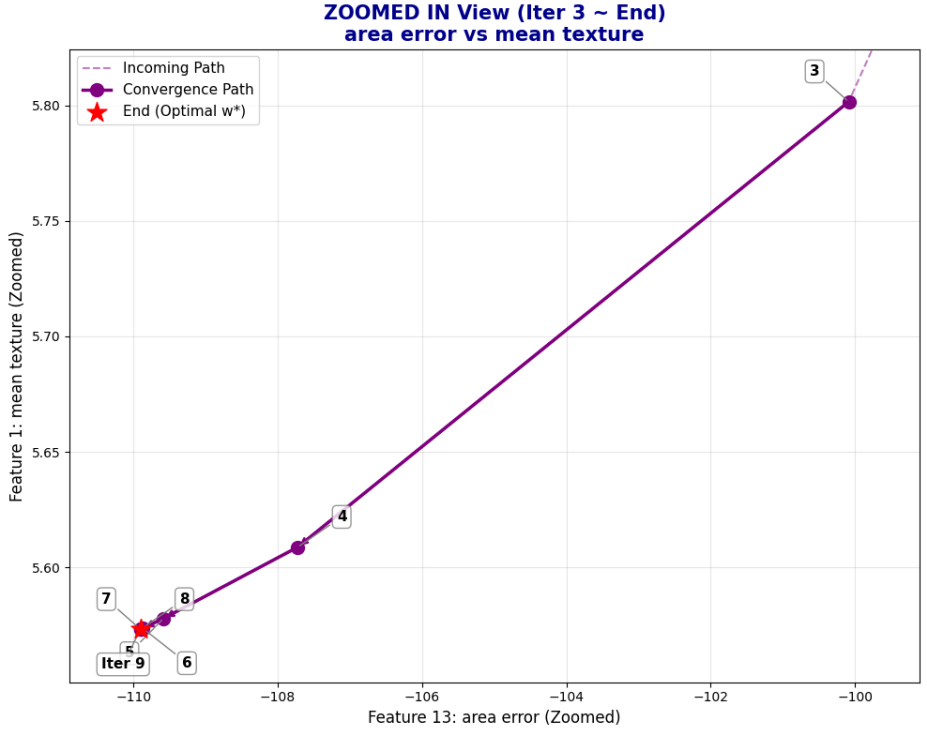
$\nabla f(w^*) =$ [-0.050194, 0, -0.046574, -0.043952, 0.188588, 0, -0, -0, 0.467301, 0, -0.000544, 0, -0.020193, -0, 0.224799, 0, -0.110369, -0, 0.224518, 0, -0, 0, -0.001577, -0.002239, 0.360005, 0.078156, -0, -0. 0.584216, 0, 0]

Step 3. 제약 솔버 실행



Step 3. 제약 솔버 실행

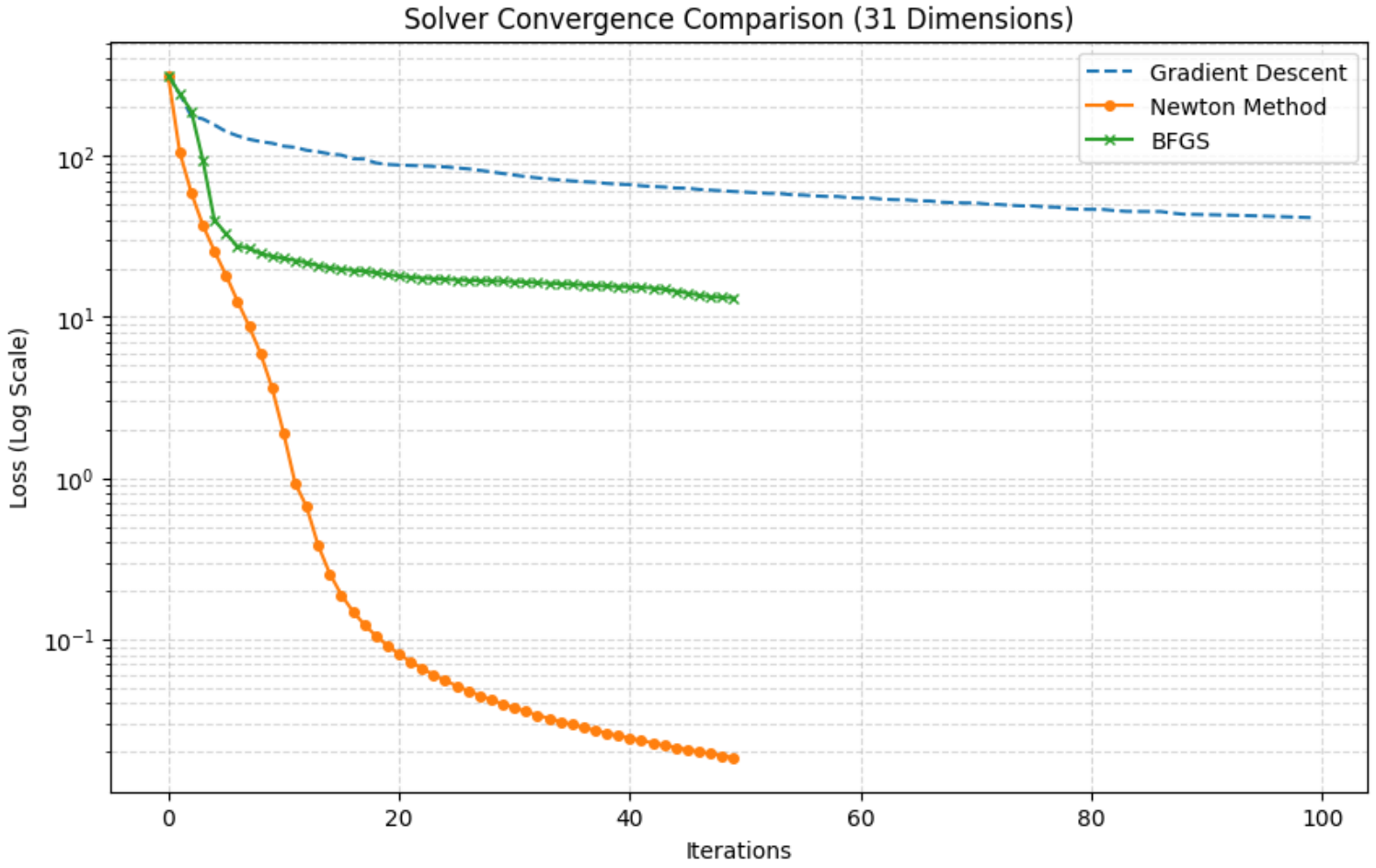




X축: Feature 13: area error (Group 1. $w \leq 0$)
Y축: Feature 1: mean texture (Group 3. no const.)

Step 4. 최적해 검증 및 분석

1. 수렴성 검증



Step 4. 최적해 검증 및 분석

2. KKT 조건 검증

1. Feasibility: $g(x^*) \leq 0, h(x^*) = 0$
2. Dual Feasibility: $\mu \geq 0$
3. Complementary Slackness: Either μ or $g(x^*)$ is zero
4. Stationarity: $\nabla f(x^*) - \sum_i \mu_i \nabla g_i(x^*) - \sum_j \lambda_j \nabla h_j(x^*) = 0$

Feature 0: [mean radius] (G1 (w〈=0))

1. $\boldsymbol{Stationarity}$:
$\nabla f(w*) = -5.02e-02, \mu\ (calc) = 5.02e-02$
$w* = -1.99e-07$
$Result = 0.00e+00\ (\approx 0, 만족: True)$

2. $\boldsymbol{Primal\ Feasibility}\ (\boldsymbol{g(w*) <= 0})$:
$g(w*) = -1.99e-07\ (<= 0, 만족: True)$

3. $\boldsymbol{Dual\ Feasibility}\ (\boldsymbol{\mu >= 0})$:
$\mu = 5.02e-02\ (>= 0, 만족: True)$

4. $\boldsymbol{Complementary\ Slackness}\ (\boldsymbol{\mu * g(w*) = 0})$:
$\mu * g(w*) = -1.00e-08\ (\approx 0, 만족: True)$

Feature 4: [mean smoothness] (G2 (w〉=0))

1. $\boldsymbol{Stationarity}$:
$\nabla f(w*) = 1.89e-01, \mu\ (calc) = 1.89e-01$
$w* = 5.30e-08$
$Result = 0.00e+00\ (\approx 0, 만족: True)$

2. $\boldsymbol{Primal\ Feasibility}\ (\boldsymbol{g(w*) <= 0})$:
$g(w*) = -5.30e-08(<= 0, 만족: True)$

3. $\boldsymbol{Dual\ Feasibility}\ (\boldsymbol{\mu >= 0})$:
$\mu = 1.89e-01\ (>= 0, 만족: True)$

4. $\boldsymbol{Complementary\ Slackness}\ (\boldsymbol{\mu * g(w*) = 0})$:
$\mu * g(w*) = -1.00e-08\ (\approx 0, 만족: True)$

Step 4. 최적해 검증 및 분석

2. KKT 조건 검증

Group 1 $(g_i = -w_i \leq 0)$: $\nabla g_i = [\dots, -1, \dots]$
Stationarity: $\nabla f_i + \mu_i(-1) = 0 \rightarrow \mu_i = \nabla f_i$
Dual Feasibility: $\mu_i = \nabla f_i \geq 0$ 인지 확인

Group 2 $(g_i = w_j \leq 0)$: $\nabla g_j = [\dots, +1, \dots]$
Stationarity: $\nabla f_j + \mu_j(1) = 0 \rightarrow \mu_j = -\nabla f_i$
Dual Feasibility: $\mu_j = -\nabla f_j \geq 0$ 인지 확인.

Group 3 (제약 없음):
$\mu_k = 0$
Stationarity: $\nabla f_k = 0$ 인지 확인.

[KKT Conditions Verification Matrix]

	Idx	Feature	Group	Stationarity	Primal Feas.	Dual Feas.	Comp. Slack.	KKT Satisfied
0	0	mean radius	G1(<=0)	True	True	True	True	True
1	1	mean texture	G3(None)	True	True	True	True	True
2	2	mean perimeter	G1(<=0)	True	True	True	True	True
3	3	mean area	G1(<=0)	True	True	True	True	True
4	4	mean smoothness	G2(>=0)	True	True	True	True	True
5	5	mean compactness	G2(>=0)	True	True	True	True	True
6	6	mean concavity	G1(<=0)	True	True	True	True	True
7	7	mean concave points	G1(<=0)	True	True	True	True	True
8	8	mean symmetry	G2(>=0)	True	True	True	True	True
9	9	mean fractal dimension	G3(None)	True	True	True	True	True
10	10	radius error	G1(<=0)	True	True	True	True	True
11	11	texture error	G3(None)	True	True	True	True	True
12	12	perimeter error	G1(<=0)	True	True	True	True	True
13	13	area error	G1(<=0)	True	True	True	True	True
14	14	smoothness error	G2(>=0)	True	True	True	True	True
15	15	compactness error	G2(>=0)	True	True	True	True	True
16	16	concavity error	G1(<=0)	True	True	True	True	True
17	17	concave points error	G1(<=0)	True	True	True	True	True
18	18	symmetry error	G2(>=0)	True	True	True	True	True
19	19	fractal dimension error	G3(None)	True	True	True	True	True
20	20	worst radius	G1(<=0)	True	True	True	True	True
21	21	worst texture	G3(None)	True	True	True	True	True
22	22	worst perimeter	G1(<=0)	True	True	True	True	True
23	23	worst area	G1(<=0)	True	True	True	True	True
24	24	worst smoothness	G2(>=0)	True	True	True	True	True
25	25	worst compactness	G2(>=0)	True	True	True	True	True
26	26	worst concavity	G1(<=0)	True	True	True	True	True
27	27	worst concave points	G1(<=0)	True	True	True	True	True
28	28	worst symmetry	G2(>=0)	True	True	True	True	True
29	29	worst fractal dimension	G3(None)	True	True	True	True	True
30	30	Bias	G3(None)	True	True	True	True	True

Step 4. 최적해 검증 및 분석

3. 정확도 (Accuracy) 평가

- Train 데이터를 솔버를 사용해 최적의 가중치 w* 찾기
- 찾은 w*를 사용하여 Test 데이터에 대한 예측 수행
- 예측값과 실제 값을 비교하여 정확도 계산

▼

평가 결과

w^* = [-0. 5.5731 -0. -0. 0. 22.4113 -4.9833 -18.1187 0.
-2.7571 -0. 13.3471 -0. -109.9035 0. 1.9433 -0. -16.3574
0. 33.3453 -22.2841 -24.5892 -0. -0. 0. 0. -3.215
-8.3867 0. -24.8532 26.3813]

Train Set Accuracy: 97.80%

Test Set Accuracy : 98.25%